

دليل البرنامج والمشروع

00

دليل البرنامج

Programme & Project Handbook — class-team-hub

المتطلبات / Requirements حاسوب + إنترنت + حساب GitHub	الدرجة الكلية / Total points 500 نقطة (+100 للوحة المتقدمة)	المخرجات / Deliverables 6 أوراق عمل + عروض الجلسات + موقع منشور	المشروع / Project class-team-hub — موقع فريق الصف	المدة الكلية / Total time 7.5 ساعة تدريب موجه	عدد الجلسات / Sessions 5 جلسات + وحدة متقدمة 06
--	--	---	--	---	--

التاريخ / Date

المجموعة / Group

اسم المستخدم / GitHub username

اسم الطالب / Student



مشروع واحد · فريق واحد · تاريخ واحد — يديره Git / one repository, one team, one history

سنبنّي معاً **موقعاً حقيقياً على الإنترنت** اسمه «دليل فريق الصف» (`class-team-hub`). كل طالب سيضيف بطاقة تعريفية له بنفسه، وسنرى البطاقات تتراكم واحداً بعد آخر حتى يصبح الموقع — الذي بدأ فارغاً — لوحةً كاملة لفريق الصف، منشورة على رابط عام يفتحه أي شخص. المشروع مكتوب بلغات الويب الأساسية (HTML و CSS و JavaScript) ولا يحتاج تثبيت أي برامج أو بيئات تطوير معقدة.

EN

Together we build a real website — “Class Team Hub” — where every student adds their own profile card, and we watch the cards accumulate until the empty site becomes a complete team board, published publicly on the internet.

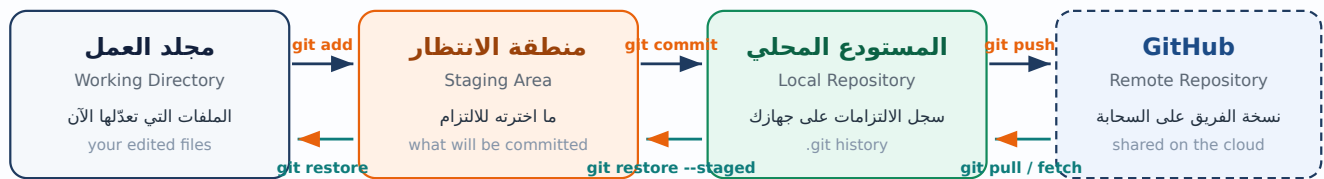
فكرة المشروع في سطر واحد

السؤال	الجواب
ما هو المشروع؟	موقع ويب بسيط يعرض بطاقة تعريفية لكل طالب في الفريق
ما نوعه تقنياً؟	موقع ثابت: HTML + CSS + JavaScript + ملفات بيانات JSON
كيف يعمل؟	<code>js/app.js</code> يقرأ بطاقات الطلبة من <code>data/members/</code> ويبني الصفحة منها
لماذا مناسب لتعلم Git؟	لأن كل طالب يضيف ملفاً خاصاً به (لا تعارض) والملف المشترك الوحيد مُوَلَّد آلياً — فیتعلّم الفريق الدمج بلا فوضى ثم يتدرب على التعارض بوعي
ما المطلوب تثبيته؟	Git + VS Code فقط (والمتصفح موجود)
ماذا سنتعلّم منه؟	الفروع · الالتزامات · طلبات السحب · المراجعة · الدمج · التعارضات · النشر

لماذا اخترنا موقعاً بلا تبعيات؟ Why a zero-dependency site?

لأن أهم ما قد يعطل ورشة Git هو مشاكل تثبيت بيئات التشغيل (`npm install` ، إصدارات Node، حزم مفقودة). اخترنا موقعاً ثابتاً ليكون تركيز الجلسات كلها على Git وGitHub — وليس على إصلاح بيئة التطوير. هذا يكسبك 40% من وقت الورشة.

خريطة البرنامج Programme map



القاعدة: لكل التزام رسالة واضحة، ولكل تعديل كبير فرع خاص.

Rule: every commit needs a clear message; every big change gets its own branch.

من جهازك إلى GitHub: add → commit → push → pull — the daily cycle

مخرجاتها	ماذا ستفعل فيها؟	الورقة موضوعها
بيئة عمل جاهزة ومؤمّنة	تثبيت Git، حساب GitHub، مفتاح SSH أو PAT، أول مستودع شخصي	01 التهيئة والتعريف
الموقع يعمل على جهازك	نسخ مستودع الفريق، قراءة بنيته، تشغيل الموقع محلياً	02 الاستنساخ والاستكشاف
بطاقتك + مراجعة مكتوبة	إنشاء فرع، كتابة بطاقتك، الالتزام، الرفع، فتح طلب سحب، مراجعة زميل	03 الفرع وطلب السحب
فهم كامل لـ merge + تقرير الفريق	جلب تعديلات الزملاء، استراتيجيات الدمج، فحص التاريخ، التراجع الآمن	04 المزامنة والدمج
موقع منشور + تسليم نهائي	صنع تعارض عن قصد وحلّه، rebase والدفع الآمن، النشر على GitHub Pages	05 التعارضات والنشر
فحص أخضر + إصدار v1.0.0	سير عمل GitHub Actions، حماية الفرع وCODEOWNERS، الإصدارات والوسوم	06 الأتمتة وحماية الجودة (وحدة متقدمة اختيارية)

دستور الفريق المختصر — يُعَلَّق على الحائط The short team constitution

1. لا عمل مباشر على `main` أبداً. كل تعديل عبر فرع، ثم طلب سحب، ثم مراجعة، ثم دمج.
2. مزامنة قبل العمل: `git switch main && git pull --ff-only` قبل إنشاء أي فرع جديد.
3. فرع واحد = مهمة واحدة. اسم الفرع: `type/username-topic` (بالإنجليزية والحروف الصغيرة).
4. رسالة التزام ذات معنى: وصف قصير `type(scope):` — لا `update` ولا `changes`.
5. لا دمج بلا مراجعة — ويكون للمراجع تعليق مكتوب محدد، لا «موافق».
6. لا تعدّل ملف زميل ولا تحذف عمله.
7. لا أسرار في المستودع: لا كلمات مرور، ولا رموز وصول، ولا بيانات شخصية حساسة.
8. افحص قبل أن ترفع: `node tests/validate-members.mjs` ثم `git diff --check`.

القاعدة	الأمر الذي يفرضها	إن خالفتها
لا عمل على <code>main</code>	حماية الفرع على GitHub (Protected branch)	الدفع يُرفض تلقائياً
رسالة التزام واضحة	<code>git commit -m "..."</code> والمراجعة البشرية	خُصم في تقييم الورقة
بيانات سليمة	<code>node tests/validate-members.mjs</code>	فشل الفحص = إعادة العمل
لا أسرار	<code>.gitignore</code> + الفحص اليدوي	خُصم أمني مباشر
مراجعة قبل الدمج	إعداد <code>Require pull request before merging</code>	لا يُدمج الطلب

أدوار الفريق (وزّعها في الجلسة الأولى) Team roles

الدور	المسؤولية	من يستفيد منه؟
قائد الفريق (1-2 لكل صف)	يدير الترتيب، يوافق على الدمج، يحلّ الخلافات التقنية	الطالب الأسرع — ليتعلّم القيادة لا الأداء فقط
مطوّر	يضيف ميزة (تحسين البحث، فترة بالوسوم، بطاقات أنيقة)	من يميل للكود
مصمّم	يحسّن الألوان والتخطيط بملف <code>css/style.css</code>	من يحب الواجهات
مختبر QA	يشغّل <code>validate-members</code> ويتحقق من ظهور البطاقات والصفحة	من يميل للتفاصيل والنقد
كاتب توثيق	يحدّث <code>README.md</code> و <code>docs/</code> وأدلة الاستخدام	من يجيد الشرح بالكتابة
مراجع (Reviewer)	يقرأ طلبات السحب ويكتب ملاحظات محددة	الجميع — ولو مرة واحدة على الأقل

توزيع الأدوار ليس «تقسيم عمل» فقط Roles are not just division of labour

اجعل كل طالب يجزّب دورين مختلفين خلال البرنامج (مثل: مصمّم ثم مراجع، مختبر ثم كاتب) حتى لا يُحصَر في تخصص واحد. المهارة المستهدفة هي الفهم الكامل لدورة العمل، لا الإتيان في زاوية واحدة منها.

ما ستقيّم عليه How you will be assessed

المكوّن	الوزن	تفصيله
المخرجات العملية (أدلة الأوامر)	40%	مخرجات حقيقية من جهازك لكل مهمة، لا نسخاً من الورقة
بطاقتك مدموجة ومراجعة زميل	20%	وجود بطاقتك على <code>main</code> مع مراجعة مكتوبة وموثقة

المكوّن	الوزن	تفصيله
حل التعارضات والنشر	20%	دليل حل تعارضين + رابط الموقع المنشور يعمل
الفهم (أسئلة التحقق وتقرير الفريق) 15%		إجابات بفهمك، وتقرير بأرقام حقيقية من المستودع
الآتمنة (اختياري – ورقة 06)	100 نقطة إضافية فحص أخضر، حالة فاشلة موثقة، إصدار مرّقم	
التعاون والاحتراف	5%	الالتزام بالاتفاقيات، جودة المراجعة، أمان بيانات الفريق

ماذا تحتاج تجهّزه قبل الجلسة الأولى؟

قائمة التحضير الشخصي / Your preparation checklist

- ☐ [] حاسوب تستطيع تثبيت البرامج عليه (لا يعمل هذا البرنامج على الجهاز اللوحي فقط)
- ☐ [] اتصال إنترنت مستقر في القاعة
- ☐ [] بريد إلكتروني فعّال (ستحتاجه لتأكيد حساب GitHub)
- ☐ [] مساحة 2 جيجابايت فارغة على القرص للتثبيت والمشاريع
- ☐ [] دفتر ملاحظات صغير – ستدوّن فيه «كل أمر جديد تعلمته وماذا يفعل»
- ☐ [] هذا الدليل + أوراق العمل الخمس

مسار مقترح لمن يريد التقدّم Optional advanced path

1. إضافة ميزة للموقع: بحث بالوسوم (tags) أو ترتيب البطاقات أو وضع ليلي – عبر فرع وطلب سحب.
2. التحقق التلقائي (CI): موجود أصلاً في المشروع (`.github/workflows/validate.yml`) – ادرسه في الورقة 06، وأضف إليه فحصاً جديداً لخطأ يتكرر في فريقك.
3. المساهمة في مشروع مفتوح: مستودع `first-contributions` أو ترجمة توثيق مشروع تحبه – بنفس الدورة التي أتقنتها: fork ← فرع ← التزام ← طلب سحب.

أهلاً بك في الفريق — لنبدأ من الورقة 01